

PDF Highlighter: Point-and-Click Field Mapping for ETL Pipelines

PDF Highlighter

Upload a PDF, draw boxes over the fields you need, and save the mapping for backend extraction.

Organization

Task Name

Form

Status

CVS Health

QWEQW

Nonimmigrant Visa Application (DS-160)

DRAFT

Cancel

Save Changes

Java Full Stack Engineer.pdf

Page 1 / 5

100%

Remove

Field mapping (3)

Coordinates are in PDF points, top-left origin — ready for pdfplumber / PyMuPDF-style cropping on the backend.

Basic Info

p1 x: 172 y: 21 w: 280 h: 63

Summary

p1 x: 39 y: 368 w: 543 h: 188

Skill

p1 x: 69 y: 111 w: 500 h: 41

JSON Payload

This is exactly what gets sent to the backend for extraction.

Mapping JSON

Copy

Download

```
{
  "label": "Basic Info",
  "page": 1,
  "x": 172,
  "y": 21,
  "width": 280,
  "height": 63
},
{
  "label": "Summary",
  "page": 1,
  "x": 39,
  "y": 368,
  "width": 543,
  "height": 188
},
{
  "label": "Skill",
  "page": 1,
  "x": 69,
  "y": 111,
  "width": 500,
  "height": 41
}
```

Click and drag on the page to draw a box around a field — draw as many as you need.

Basic Info

Nabeel Ahmed Jamil

Lead Full Stack Engineer

Email: nabeelajwa93@gmail.com | Ph: +1(630)-259-6984

Professional Summary

Over 11+ years of experience as a Senior Full Stack and Distributed Systems Engineer delivering enterprise applications across Banking, Telecom, Healthcare, FinTech, and Ad-Tech domains. Expertise in designing cloud-native microservices, event-driven architectures, and high-throughput distributed systems using Java, Spring Boot, Angular, React, and Python.

Extensive experience building scalable solutions on AWS and Azure using Kubernetes (EKS/AKS), Docker, AWS Lambda, Azure Functions, Step Functions, Kafka, Amazon SQS, SNS, EventBridge, MQ, and Azure Service Bus.

Strong expertise in REST, SOAP, GraphQL, Microservices, Spring Boot, Spring Cloud, Spring Security, Hibernate, JPA, OAuth 2.0, OpenID Connect (OIDC), JWT, Amazon Cognito, Azure AD, and enterprise integration.

Hands-on experience in Python backend development, automation, ETL, OCR, and AI using FastAPI, SQLAlchemy ORM, Pydantic, OpenCV, Tesseract, Pandas, NumPy, BeautifulSoup, Selenium, Playwright, and Whisper.

Experienced in building AI-powered applications using LangChain, LangGraph, Retrieval-Augmented Generation (RAG), Model Context Protocol (MCP), OpenAI, Claude, Vector Databases, and LLM integrations for intelligent search, document processing, and workflow automation.

Proficient in AI-assisted software development using GitHub Copilot, OpenAI Codes, Cursor, and Claude Code for code generation, refactoring, testing, debugging, documentation, and productivity improvements.

Strong experience with PostgreSQL, Oracle, DB2, MongoDB, Amazon Aurora, Azure SQL CI/CD using GitHub Actions, Jenkins, AWS CodePipeline, Azure DevOps, Infrastructure as Code with Terraform, and container orchestration using Docker and Kubernetes.

Experienced in monitoring and observability using CloudWatch, ELK/OpenSearch, Prometheus, Grafana, and AWS X-Ray, delivering secure, scalable, cloud-native, and AI-enabled enterprise solutions in Agile environments.

Technical Skills

Languages: Java (8-17), Python, JavaScript, TypeScript

Backend & Frameworks: Spring Boot, Spring MVC, Spring Security, Spring Batch, Spring Data JPA, Hibernate, Struts, REST APIs

Frontend: Angular 17+, Angular Material, PrimeNG, HTML5, CSS3, JavaScript, jQuery, Ag-Grid, Storybook

PDF Highlighter — drawing field boxes over a resume and generating the JSON payload

The problem with "just OCR the whole PDF"

Most ETL pipelines that ingest PDFs — resumes, visa applications, invoices, insurance forms — start the same way: run the file through OCR or a text extractor, dump the whole page into an LLM prompt, and ask it to "find the name, email, and summary." It works, until it doesn't:

- **Noise drowns the signal.** A one-page resume can have 40+ lines of text. Asking a model to guess which lines are "Basic Info" versus "Skills" versus a stray footer works most of the time and fails silently the rest of the time.
- **Cost scales with the whole page, not the answer.** Every extraction re-reads and re-reasons over content you don't need, on every single document.
- **Nothing is reproducible.** If today's extraction pulled the wrong paragraph into "Summary," there's no record of *why* — just a wrong field in the output.
- **Every new document layout means new prompt tuning.** A visa form (DS-160) and a resume don't share a shape, so a single "extract everything" prompt has to work for both, which means it works well for neither.

What PDF Highlighter does instead

PDF Highlighter flips the extraction model: instead of asking software to *guess* where the data lives, a person draws it — once, per template — and the coordinates become the extraction contract.

The workflow, based on the tool as built:

- 1 **Set up a task.** Pick an organization, give the task a name, attach it to a form (e.g. "Nonimmigrant Visa Application (DS-160)"), and upload the source PDF. A task carries a lifecycle status — DRAFT while you're still marking fields, READY once the mapping is finalized for downstream use.
- 2 **Draw boxes over the fields you care about.** Click-and-drag directly on the rendered PDF page (via `pdf.js`) to mark a region — a name block, a paragraph, a table. Each box gets a label ("Basic Info," "Summary," "Skill," etc.), and you can reorder, rename, or delete boxes as you refine the mapping.
- 3 **Coordinates are captured in PDF points, top-left origin** — the same coordinate space used by `pdfplumber` and `PyMuPDF` on the backend. That's a deliberate choice: the frontend doesn't just record "a box on screen," it records a crop rectangle the backend can apply directly against the source PDF with zero coordinate translation.
- 4 **Save the mapping.** Fields are persisted per task (page, x, y, width, height, label, `displayOrder`), and the frontend reconciles adds/edits/deletes against the server on save — so a mapping can be revised over time without losing history of what changed.

Here's the actual payload shape a saved task produces, from the resume example — three boxes on page 1:

```
[
  {
    "label": "Basic Info",
    "page": 1,
    "x": 172,
    "y": 21,
    "width": 280,
    "height": 63
  },
  {
    "label": "Summary",
    "page": 1,
    "x": 39,
    "y": 368,
    "width": 543,
    "height": 188
  },
  {
    "label": "Skill",
    "page": 1,
    "x": 69,
    "y": 111,
    "width": 500,
    "height": 41
  }
]
```

That's the entire extraction instruction set. No prompt, no heuristics — just "crop this rectangle on this page and read what's inside."

How this plugs into an ETL pipeline

Map it onto the standard Extract → Transform → Load stages:

Extract. The backend takes the saved (page, x, y, width, height) list and crops exactly those regions out of the source PDF (pdfplumber/PyMuPDF-style), running text extraction or OCR *only* on those crops — not the full page. For a batch of documents that share a template (every DS-160 submission, every resume built off the same export tool), the same mapping is reused across every file: draw it once, apply it to thousands.

Transform. Each label becomes a key in a structured record — label → extracted_text. Because the label is human-assigned and stable, mapping it to a target schema field ("Basic Info" → candidate.header, "Summary" → candidate.summary) is a simple, deterministic lookup instead of an LLM classification step.

Load. Since the output at this point is just labeled key/value data, it's trivial to fan it out into whatever the destination needs — a JSON document for an API consumer, a row in a CSV/XLSX export, a record written to a database table, or values used to auto-fill a downstream form (fitting, given the tool is itself scoped to a specific form per task).

Why targeted extraction beats "read all the data"

- **Precision over guessing.** A bounding box is unambiguous. There's no scenario where the model "reads the wrong paragraph" — it physically can't see anything outside the box.
- **Cost and speed.** Cropping to a handful of small regions and running extraction only there is cheaper and faster than OCR/LLM reasoning over an entire multi-page document, especially at batch scale.
- **Template reuse.** Documents generated from the same source (a visa form, a resume builder, an invoice template) have a fixed layout. One mapping, defined visually in minutes, extracts every future document of that shape — no re-prompting per file.
- **Auditability.** Because the mapping is stored as explicit coordinates tied to a labeled field, anyone can open the task, see the box on the actual page, and verify *exactly* what produced a given value. If an extraction looks wrong, you fix the box, not a prompt.
- **Format independence.** The extraction layer only produces labeled text. Whether that ends up as JSON, a spreadsheet row, or a database record is a downstream decision — the hard, error-prone part (finding the data) is solved once, upstream of all of those formats.

The takeaway

PDF Highlighter isn't trying to "understand" a PDF the way an LLM reading raw text does — it's encoding human judgment about *where* the data lives, once, as reusable geometry. That trade — a few minutes of manual box-drawing per template — buys deterministic, cheap, auditable extraction for every document that follows the same layout, which is exactly the kind of guarantee an ETL pipeline needs and a "read everything and hope" approach can't give you.